

AI-enabled Digital Twin Platform for Aero Piston Engines in MALE UAV Missions

Detailed project overview

Problem Statement ID: 26054 | Repository branch: Yash | Generated: September 2026

Purpose

Unify synthetic telemetry, physics-based digital-twin state estimation, machine-learning intelligence, backend services, and an operational frontend into one demonstrable platform.

1. Executive Summary

DHRUVA is a representative prototype for monitoring the health of aero piston engines used in medium-altitude, long-endurance unmanned aerial vehicle missions. It models expected engine behavior from operating conditions, compares expected values with observed telemetry, converts the resulting residuals into component health and an overall Health Index, and exposes the resulting information to an operational web console.

The repository is organized as a full-stack engineering demonstrator. The root implementation contains Python simulation, digital-twin, machine-learning, backend, database, streaming, agent, and test layers. The React/Vite frontend presents fleet, engine, mission, fault, tuning, and maintenance workflows. Several DHRUVA-* directories are parallel team snapshots or specialized copies; the root folders are the primary integrated surface used by this report.

- Phase 1 generates reproducible, labelled synthetic telemetry under configurable engine, mission, environment, sensor-noise, and fault conditions.
- Phase 2 runs telemetry sequentially through a physics-inspired digital twin and produces residuals, validity flags, component health, and a smoothed Health Index.
- The ML layer adds cleaning, feature engineering, anomaly detection, fault classification, RUL estimation, sensor-drift detection, trend forecasting, and explainability.
- The FastAPI backend provides REST endpoints for engines, telemetry, health, faults, missions, tuning, replay, reports, alerts, subsystems, agent interactions, and fleet monitoring.
- WebSockets provide engine-specific or fleet-wide telemetry streams for live frontend updates.

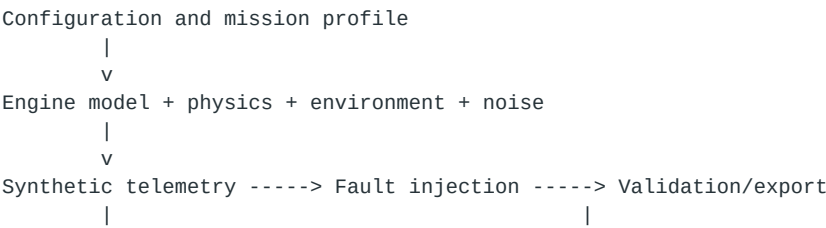
2. Goals, Scope, and Safety Position

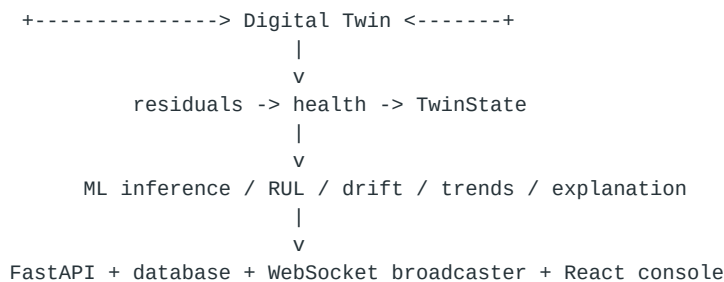
The system is designed to make engine behavior observable and analyzable before real or certified data is available. Its central value is an explainable chain from operating conditions to expected physics, residual evidence, health scoring, and operator-facing action. The project is not presented as an aircraft safety system or as a substitute for certified engine maps, FADEC logic, maintenance procedures, or operational limits.

In scope	Out of scope or explicitly limited
Synthetic telemetry generation; mission and engine configuration; representative fault injection; data export; physics-inspired residuals; health scoring; optional ML inference; backend APIs; persistence; WebSocket streaming; web console; test fixtures.	Classified or proprietary DRDO data; certified limits; thermodynamic cycle fidelity; operational aircraft control; production-grade sensor-error characterization; validated maintenance intervals; safety certification.
Primary users	Expected decisions
Prototype developers, data scientists, backend/frontend engineers, mission analysts, and maintenance-planning demonstrators.	Inspect fleet and engine state, identify abnormal behavior, compare missions, review fault evidence, estimate remaining life, and schedule or review maintenance actions.

3. System Architecture

At a high level, DHRUVA is a pipeline with two complementary loops: a batch/replay loop for reproducible simulation and analysis, and a service loop for ingestion, persistence, streaming, and operator interaction.





The backend application is created in `backend/app/main.py`. Its lifespan initializes database tables/seeds and registers the telemetry WebSocket broadcaster. The API router is mounted under both `/api` and `/api/v1`; the WebSocket paths are `/ws/telemetry/{engine\_id}` and `/ws/telemetry`.

Layer	Root location	Responsibility
Simulation and data	simulation/, data/, configs/	Generate engine telemetry, missions, noise, faults, validation, and CSV/JSON exports.
Digital twin	digital_twin/	Expected-vs-actual comparison, residuals, validity, component health, smoothing, and TwinState.
ML intelligence	ml/	Prepare data, engineer features, train or load models, infer anomaly/fault/RUL/trends, and explain results.
Backend	backend/app/	FastAPI services, schemas, database models, middleware, REST endpoints, and application lifecycle.
Persistence	database/	SQL schema, migrations, seeds, and storage concepts for fleet and operational records.
Streaming	streaming/	WebSocket manager and telemetry broadcast integration.
Frontend	frontend/	React/Vite command console, routing, context, charts, pages, controls, and live-state presentation.
Agent	agent/	Prompt, schema, and tool surfaces for an AI copilot capability.

4. Phase 1: Synthetic Telemetry and Simulation

Phase 1 is the data foundation. It produces reproducible telemetry that has enough structure for digital-twin residuals and ML development, while clearly documenting that the coefficients are representative and not certified engine maps.

Canonical telemetry includes identifiers and context such as timestamp, engine ID, model ID, mission ID, altitude, ambient temperature, throttle, and mission phase. It also includes RPM, CHT, EGT, oil pressure and temperature, fuel flow, vibration RMS, battery voltage, alternator current, injection timing, fault labels, fault activity/severity, quality/status fields, simulation seed, Health Index, RUL, and residual columns.

Simulation concern	Current behavior
Engine classes	O320_CLASS (~100 HP class), ROTAX914_CLASS (~115 HP class), and AE300_CLASS (~170 HP class).
Mission profiles	HIGH_ALTITUDE_ISR, LONG_ENDURANCE_LOITER, RAPID_RESPONSE, HOT_WEATHER, MARITIME_PATROL, and CUSTOM, with TAKEOFF, CLIMB, CRUISE, LOITER, DESCENT, and LANDING phases.
Environment	Altitude reduces density ratio and affects power/cooling; ambient temperature changes cooling effectiveness and thermal channels.
Sensor behavior	Per-channel Gaussian noise is configured per engine; data quality and sensor status are carried in the schema.
Faults	Injector degradation, combustion disturbance, cooling degradation, lubrication pressure loss, overheating, abnormal vibration, alternator/battery degradation, sensor bias/drift/dropout.
Fault timing	Faults support start, duration, severity, gradual or sudden progression, affected sensor, and multiple active faults.
Exports	CSV and JSON outputs under data/synthetic/healthy, faults, and missions; large generated artifacts are intended to be ignored by Git.

The documented healthy relationships include lagged RPM, load and fuel flow relationships, thermal channels, oil temperature/pressure, vibration, electrical output, and injection timing. The public simulation entry point is `simulation.generate_telemetry`, also exported by `simulation/__init__.py`.

Example commands

```
pip install -r requirements.txt
pytest
python generate_demo.py
python data/generate_dataset.py --engine O320_CLASS --mission HIGH_ALTITUDE_ISR --duration 120 --stem dem
```

5. Phase 2: Digital Twin and Health Intelligence

`digital_twin.twin.DigitalTwin` is a sequential stateful processor. For every telemetry record it reconstructs expected values from altitude, throttle, ambient temperature, previous expected RPM, previous expected oil temperature, and the selected engine parameters. It then compares actual values with expected values rather than copying measurements into the expected side.

```
telemetry sample
-> expected_from_operating(...)
-> residual = actual - expected
-> normalized residuals and validity flags
-> component_scores(...)
-> EMA smoothing of components and overall index
-> health state + TwinState
```

The monitored channels cover RPM, CHT, EGT, oil pressure, oil temperature, fuel flow, vibration, battery voltage, and alternator current. Component groups are thermal, lubrication, mechanical, combustion, electrical, and

sensor/data quality. Health scoring uses exponential penalties based on absolute normalized residuals, combines component scores with a worst-component influence, and applies configurable exponential moving average smoothing.

Health index	Demonstration state
90-100	NORMAL
70-89	WATCH
40-69	WARNING
0-39	CRITICAL

Validity is deliberately separate from scoring. A record can carry data-quality or missing-channel flags while the scoring pipeline still produces a state. Optional MLResult input can lower the index, but Phase 2 does not create ML predictions by itself. ``process_dataframe`` preserves chronological order and returns a list of TwinState objects; states can be flattened and exported.

6. Machine-Learning and Analytics Layer

The ML layer is designed as a reusable inference stack rather than a single model. ``ml.inference.inference_service.run_inference`` cleans incoming telemetry, engineers temporal and residual features, extracts twin state when present, attempts trained models when artifacts exist, and falls back to deterministic heuristics when they do not.

Capability	Role in the pipeline
Cleaning	Normalize column names, validate required fields, remove duplicates, identify telemetry/twin schemas, and preserve inference consistency.
Feature engineering	Create rates, rolling features, cross-sensor relationships, physics-prediction residuals, and twin residual features.
Anomaly detection	Estimate whether the current operating window is abnormal and produce score/severity.
Fault classification	Estimate a fault type and confidence for conditions such as cooling, lubrication, vibration, sensor, or electrical issues.
RUL	Use a trained model when available, otherwise map Health Index and anomaly evidence to a prototype remaining-life estimate with uncertainty.
Sensor drift	Look for persistent bias or drift patterns that can be distinguished from engine faults.
Trend forecasting	Forecast short-horizon sensor or health trends from the prepared series.
Explainability	Use model explanations when possible; otherwise report high-magnitude telemetry factors as a transparent fallback.

The final inference payload contains engine and timestamp context, Health Index and status, anomaly detection, fault type/confidence, RUL information, drift, trends, explanation factors, and related state. The service explicitly catches unavailable or failing model artifacts and falls back to heuristics, which makes demos resilient but also means production deployments must surface model provenance and fallback status to operators.

7. Backend API and Persistence

The FastAPI application is the integration boundary between telemetry producers, persistence, digital-twin/ML services, and the frontend. CORS, request logging, centralized error handling, database initialization, and OpenAPI documentation are wired at application startup.

API group	Representative routes and purpose
Health	<code>`GET /api/health`</code> for system health and readiness information.
Engines	List/create engines, list models, retrieve engine details, and retrieve engine telemetry.
Telemetry	Ingest telemetry and retrieve historical data.
Fleet	Fleet summary and fleet telemetry views.
Faults and alerts	Review fault records and operator alert state.
Missions	List, create, and retrieve active/current mission information; <code>`/api/mission`</code> bridges the frontend singular route.
Twin and subsystems	Health, subsystem conditions, and digital-twin-derived operational status.
Tuning	Retrieve candidates and create tune profiles for calibration workflows.
Replay	Replay or inspect historical telemetry scenarios.
Reports and maintenance	Generate engine reports and synchronize MRO/maintenance information.
Agent	Expose the AI copilot integration surface.

The SQL schema defines tables for engine models, engines, telemetry, twin states, faults, missions, tune profiles, and maintenance. Foreign keys preserve relationships from an engine to its model and operational records. Time-oriented indexes support engine-plus-timestamp queries for telemetry, twin states, and faults.

The schema shown in ``database/schema.sql`` uses PostgreSQL-oriented types such as `BIGSERIAL` and `JSONB`. Runtime database settings and migrations should be treated as deployment configuration; the repository also contains seeds and migration folders for environment-specific initialization.

8. Streaming and Live Operation

The backend registers ``TelemetryService`` with a WebSocket manager during application startup. Clients can connect to an engine-specific stream at ``/ws/telemetry/{engine_id}`` or subscribe to all engines at ``/ws/telemetry``. The manager tracks connections and broadcasts telemetry to the appropriate audience. The frontend's `EngineContext` owns stream state and exposes a toggle used by the shell/sidebar.

A production version should define message envelopes, authentication, reconnect/backoff behavior, heartbeat semantics, ordering guarantees, replay boundaries, and backpressure policy explicitly. The current endpoint keeps the connection open and receives optional client text messages while the server-side telemetry service performs broadcasts.

9. Frontend Operational Console

The frontend is a React 19 and Vite application using TypeScript, React Router, Recharts, Motion, Tailwind CSS, and Lucide icons. The application shell has a fixed tactical sidebar, a command header, an aerospace-grid main viewport, authentication context, engine context, and an error boundary.

Route/View	Operational purpose
Entry and Login	Authentication and entry flow.
Executive Overview	High-level command and fleet context.
Dashboard	Primary operational dashboard for health, alerts, and live signals.
Engine Details	Per-engine telemetry, health, and subsystem information.
Fault Diagnostics	Fault evidence and root-cause-oriented diagnostics.
Mission Simulation	Mission scenario and telemetry simulation sandbox.
Mission Tuning	Digital-twin mission tuning and calibration workflow.
Fleet Monitoring	Fleet-wide airspace and engine monitoring view.
Maintenance	Predictive maintenance and MRO scheduling workflow.

The documented UI direction is a dark aerospace command-console/HUD language with cyan, amber, red, and green status semantics. The frontend package exposes ``npm run dev``, ``npm run lint``, ``npm run build``, and ``npm run preview``. A design guidance document in ``docs/`` emphasizes preserving routes, data flow, WebSocket behavior, responsive behavior, and existing controls while improving spacing and scanability.

10. Agent and Copilot Surface

The ``agent/`` package contains an agent entry point, prompts, schemas, and tools. The backend includes an AI Copilot Agent API router. Together these indicate an intended conversational layer for querying operational state or assisting with analysis. The agent should be treated as an interpretation and workflow-assistance surface, not as an autonomous authority for flight or maintenance release decisions.

Recommended contract boundary: the agent should consume typed, timestamped, provenance-aware facts from the backend and return citations to the underlying engine, telemetry, twin state, fault, and maintenance records. Any action that changes configuration, tuning, or maintenance status should remain explicit and auditable.

11. Testing and Quality Signals

The root tests cover the main simulation and ML contracts. Simulation tests verify engine model loading, mission profile loading, schema shape, reproducibility from seeds, environment effects, engine differences, fault labels, gradual/sudden progression, sensor drift/dropout, invalid parameters, export, and correlated channels. Twin tests verify healthy state, thermal/lubrication/vibration effects, sensor drift, sudden faults, and gradual degradation. ML tests cover cleaning, feature engineering, preprocessing consistency, anomaly and fault models, RUL, trends, explainability, sensor drift, and inference contracts.

Check	Command or evidence
Python test suite	<code>`pytest`</code> from the repository root.
Demo generation	<code>`python generate_demo.py`</code> writes representative healthy/fault/missions artifacts.
ML demonstration	<code>`python ml/run_demo.py`</code> compares a healthy window with a cooling-degradation sequence.
Frontend type/lint check	<code>`cd frontend; npm run lint`</code> runs TypeScript no-emit checking.
Frontend production build	<code>`cd frontend; npm run build`</code> runs TypeScript and Vite build.
Backend API visibility	FastAPI OpenAPI at <code>`/openapi.json`</code> , Swagger UI at <code>`/docs`</code> , ReDoc at <code>`/redoc`</code> .

A useful next quality step is an end-to-end test that starts the backend, ingests one generated telemetry window, verifies persistence, consumes a WebSocket message, runs inference, and confirms the frontend-facing response contract. This would validate the integration seams currently covered mostly by unit and component-level tests.

12. Local Runbook

Python environment

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest
python generate_demo.py
```

Backend

```
uvicorn backend.app.main:app --host 0.0.0.0 --port 8000
# API docs: http://localhost:8000/docs
```

Frontend

```
cd frontend
npm install
npm run dev
```

Vite development server is configured for port 3000

Containers and deployment

The repository contains root, backend, deployment, and team-specific Docker Compose/Dockerfile surfaces. Before using a compose file, verify which copy is intended for the target environment, confirm database credentials and connection strings, and check that frontend API/WebSocket base URLs match the deployment topology.

13. Data and Model Lifecycle

Stage	Inputs	Outputs
Configure	JSON under configs/ for engines, missions, faults, simulation, twin health.	Validated named configurations.
Simulate	Engine model, mission, duration, sampling rate, seed, optional faults.	Telemetry DataFrame and CSV/JSON datasets.
Twin	Ordered telemetry records and optional MLResult.	Residuals, component health, Health Index, state, validity flags.
Train	Clean telemetry, labelled faults, twin features, configured feature columns.	Model artifacts, preprocessors, metrics, and model metadata.
Infer	Telemetry window, available twin state, optional trained artifacts.	Anomaly, fault, RUL, drift, trends, explanations, status.
Persist	Telemetry, engine, mission, twin, fault, tuning, report, maintenance records.	Queryable operational history.
Present	REST responses, WebSocket events, context state.	Dashboards, diagnostics, simulation, fleet, and maintenance views.

Reproducibility is a strong design feature: simulation accepts a seed, outputs carry ``simulation_seed``, and ML tests use in-memory fixtures and temporary artifact paths. For deployment, add model version, training dataset identifier, configuration hash, and inference timestamp to every derived result so operators can reconstruct why a state was produced.

14. Strengths and Engineering Tradeoffs

- Strong modular decomposition: simulation, twin, ML, backend, persistence, streaming, frontend, and agent concerns are separated.
- Physics-inspired residuals provide an interpretable reference signal and make synthetic fault behavior more realistic than independent random columns.
- The sequential twin retains lagged state, allowing operating history to influence expected RPM and oil temperature.
- ML inference degrades gracefully when model artifacts are unavailable, which supports demos and incremental development.
- The API and database shapes cover the main operational objects needed for an end-to-end demonstrator.
- The main tradeoff is prototype fidelity: representative coefficients, tunable thresholds, heuristic fallbacks, and synthetic data are useful for development but cannot establish real-world safety or maintenance decisions.

15. Limitations and Production Readiness Gaps

- The simulation is algebraic and physics-inspired, not a thermodynamic cycle model or certified FADEC model.
- Engine coefficients and health bands are demonstration values; they need calibration against traceable test-cell and flight data.
- Synthetic Gaussian noise does not represent the full error, latency, dropout, calibration, or wiring behavior of real sensors.
- RUL estimates need validated survival/degradation targets, uncertainty calibration, censoring treatment, and maintenance-label governance.

- Production streaming requires authentication, authorization, reconnect policy, message schemas, ordering, backpressure, and observability.
- Database migrations, seed behavior, secrets, backups, retention, and tenancy need environment-specific operational hardening.
- Model fallbacks should be visible in the UI and logs, with model/data/config provenance attached to every prediction.
- Safety-critical use would require independent verification, validation, hazard analysis, human authority, and applicable aviation certification processes.

16. Recommended Evolution Path

Priority	Recommended next step	Why it matters
1	Define canonical API/event schemas and generated TypeScript/Python clients.	Prevents drift between backend, ML, streaming, and frontend contracts.
2	Add an end-to-end telemetry-to-dashboard integration test.	Validates the complete operational path, not only isolated modules.
3	Attach provenance and fallback metadata to twin/ML results.	Lets operators distinguish trained-model output from heuristic demo output.
4	Calibrate engine and mission models with governed reference data.	Moves from illustrative behavior toward useful engineering estimates.
5	Harden WebSocket and database operations.	Improves reliability, security, replay, recovery, and fleet-scale behavior.
6	Establish RUL evaluation protocols and uncertainty reporting.	Makes predictive maintenance claims measurable and auditable.
7	Add role-based approval and audit trails around tuning and maintenance actions.	Keeps human authority explicit for consequential workflows.

17. Repository Map

DHRUVA/ agent/ backend/app/ configs/ data/ database/ digital_twin/ docs/ frontend/ ml/ simulation/ streaming/ tests/ docker-compose.yml pyproject.toml	AI copilot entry point, prompts, schemas, tools FastAPI app, routes, services, models, schemas engine, mission, fault, safety, simulation, twin JSON schemas, generators, raw/processed/synthetic data schema, migrations, seeds engines, physics, residuals, health, state, validity phase docs, architecture and UX guidance React/Vite operational console preprocessing, models, inference, RUL, trends, drift engine simulator, missions, config loader, paths WebSocket manager and stream integration simulation, twin, ML, and backend tests root orchestration entry point Python package metadata and dependencies
--	---

Conclusion

DHRUVA's core contribution is an explainable, end-to-end prototype: configurable mission-aware telemetry becomes expected physics, residual evidence, health state, ML interpretation, persistent operational records, live streams, and an operator console. Its architecture is already shaped for incremental maturation. The essential next step is disciplined validation and provenance: establish which outputs are simulated, heuristic, trained, calibrated, or

certified, and make that distinction visible wherever a human might act on the result.

Source basis: repository files and documentation available at generation time. This document is descriptive and does not add claims beyond the implementation and project notes inspected.